TP3 - Projet Cloud Computing

Analyse de sentiment des commentaires YouTube

Questions de réflexion

Élément de Module : Industrie 4.0

Réalisé par : Ilyas Fardaoui

Encadré par : Prof. Maria Zemzami

Université Mohamed V - ENSAM Rabat
Filière INDIA | Année universitaire 2026/27

PHASE 1

Phase 1 : Configuration de l'environnement MLOps

1 - Pourquoi séparer les données raw et processed ?

Le dossier data/raw conserve les données brutes intactes : si le script de nettoyage change (gestion de la ponctuation, des symboles), on régénère data/processed sans retélécharger ni altérer la source.

Cela garantit un flux unidirectionnel (raw -> nettoyage -> processed) et permet à l'entraînement de charger directement les splits nettoyés sans rejouer le nettoyage à chaque expérimentation.

2 - Quels fichiers ne doivent jamais être commités sur Git ?

L'environnement virtuel (.venv/), les secrets (.env, tokens), les datasets volumineux (data/raw et data/processed/*.csv), les caches et artefacts (__pycache__/, *.pyc, logs/), ainsi que les gros modèles. Mon .gitignore couvre précisément ces cas.

3 - Quelle est l'importance d'un environnement virtuel ?

Il isole les dépendances du projet (versions de scikit-learn, FastAPI, etc.) pour éviter les conflits entre projets, et protège l'installation Python globale du système. Il rend aussi l'environnement reproductible via requirements.txt.

PHASE 2

Phase 2 : Pipeline de données et préparation

Après nettoyage : 36 562 commentaires (train 29 249 / test 7 313). Répartition : positif 15 723 (43,0 %), neutre 12 606 (34,5 %), négatif 8 233 (22,5 %) — plus de 1 000 par classe comme exigé.

1 - Pourquoi nettoyer les URLs et mentions ?

Les URLs et mentions sont quasi uniques et créent des milliers de tokens rares, sans valeur sémantique, qui gonflent la matrice TF-IDF. Les remplacer par des tokens (URL, USER, HASHTAG) réduit le bruit et la dimensionnalité et limite le surapprentissage, ce qui aide le modèle à généraliser aux commentaires YouTube.

2 - Comment gérer les emojis dans le texte ?

Dans mon nettoyage actuel, la normalisation (suppression des caractères non alphanumériques hors ! ? ') retire les emojis avec les autres symboles. C'est un choix de simplicité qui privilégie le signal textuel. Une amélioration identifiée serait de les convertir en tokens sémantiques (emoji_positif, emoji_négatif) avant la normalisation pour conserver ce signal émotionnel, utile sur les commentaires courts.

3 - Quel ratio train/test est approprié ?

Un split stratifié 80 % / 20 % (random_state=42) : environ 29 249 commentaires pour apprendre le vocabulaire TF-IDF et la validation croisée, et 7 313 pour une évaluation finale fiable (dont plus de 1 600 pour la classe minoritaire négative). La stratification préserve la distribution des classes.

PHASE 3

Phase 3 : Développement et entraînement du modèle

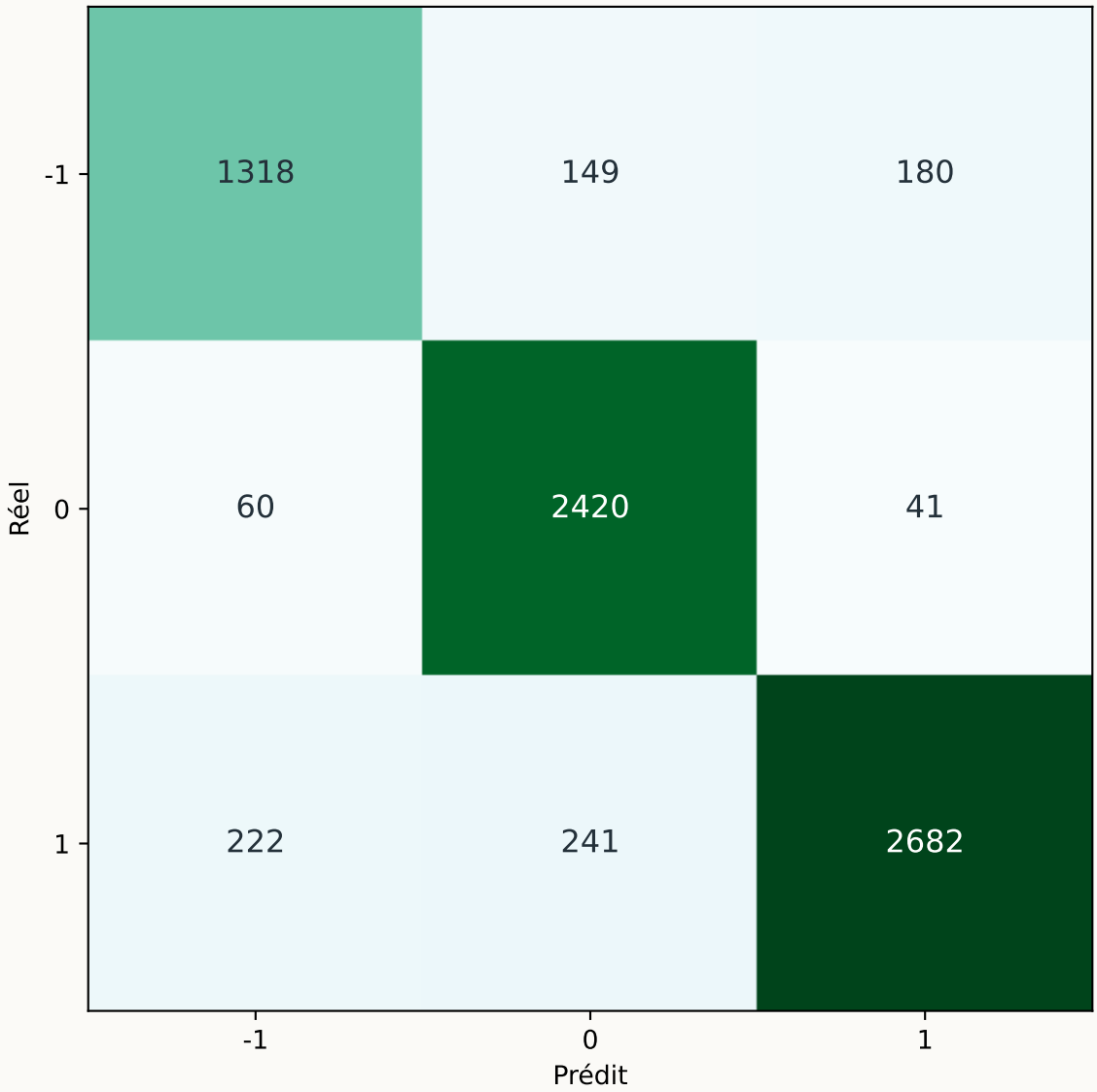


Figure 3.1 : Matrice de confusion (jeu de test, 7 313 commentaires).

PHASE 3

Phase 3 : Comparaison des modèles

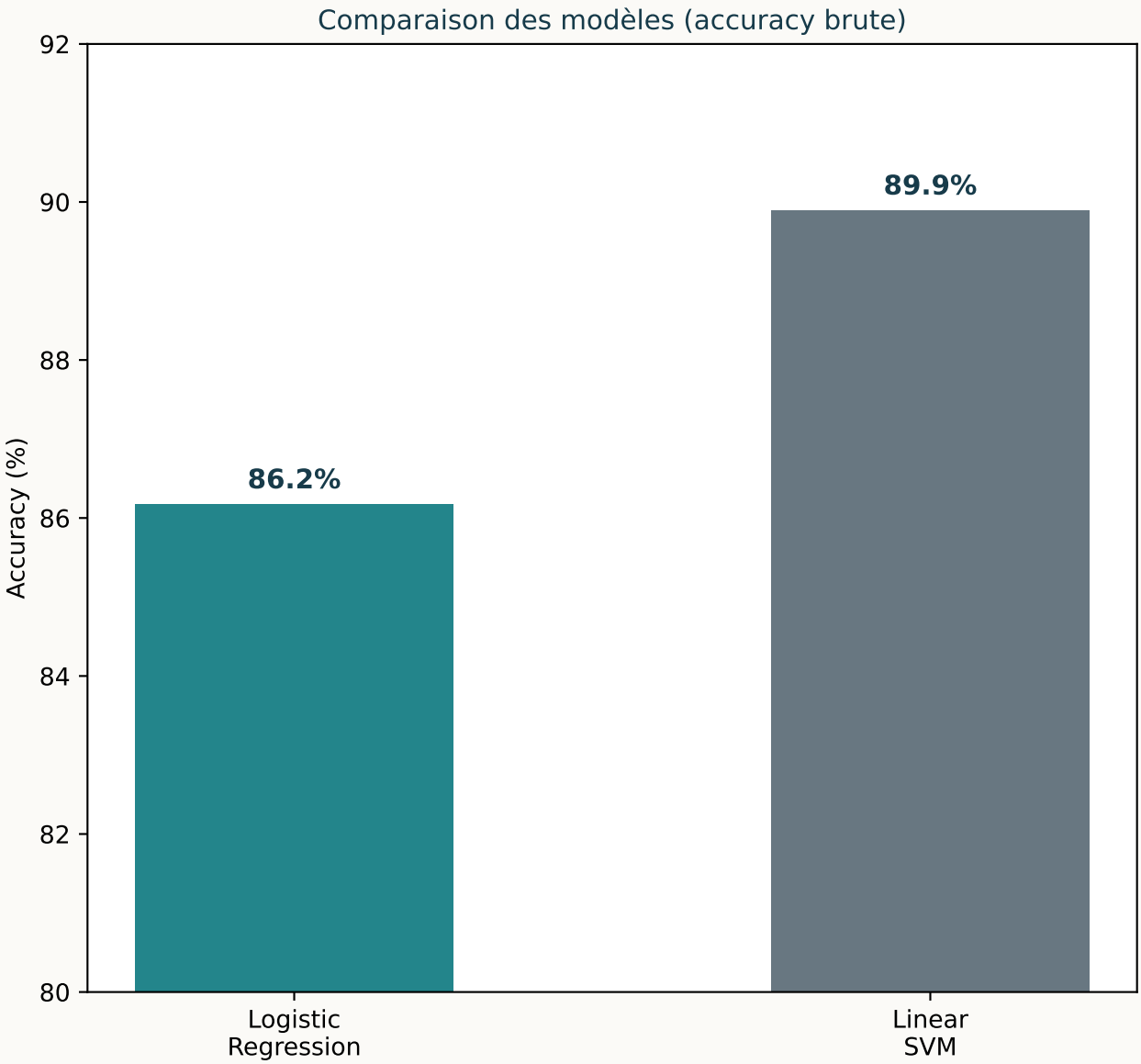


Figure 3.2 : Logistic Regression retenue pour ses probabilités (predict_proba).

PHASE 3

Phase 3 : Questions de réflexion

1 - Pourquoi utiliser TF-IDF plutôt que Count Vectorizer ?

CountVectorizer donne un poids élevé aux mots fréquents mais peu informatifs (the, video, think). TF-IDF pondère la fréquence locale par la rareté globale (IDF) et met en valeur les termes discriminants porteurs d'opinion (amazing, horrible, waste).

2 - Comment interpréter la matrice de confusion ?

La diagonale (1318 + 2420 + 2682 = 6420 sur 7313, soit 87,79 %) représente les prédictions exactes. La classe Neutre a le meilleur rappel (96,0 %), devant Positif (85,3 %) et Négatif (80,0 %). Les confusions principales opposent négatif et positif aux neutres, souvent sur des commentaires ambigus ou ironiques.

3 - Quels hyperparamètres ont le plus d'impact ?

GridSearchCV a retenu C=2.0, min_df=2, ngram_range=(1, 1). class_weight='balanced' corrige le déséquilibre de la classe négative (F1 porté à 81,18 %). Le paramètre C ajuste la régularisation L2 et ngram_range contrôle la prise en compte des expressions composées.

PHASE 4

Phase 4 : Développement de l'API REST

1 - Pourquoi utiliser FastAPI plutôt que Flask ?

FastAPI est asynchrone (ASGI/Uvicorn) et plus rapide que Flask (WSGI) pour servir des prédictions en temps réel. Il valide nativement les entrées avec Pydantic (erreurs 422 explicites) et génère automatiquement la documentation OpenAPI (Swagger /docs).

2 - À quoi sert la configuration CORS ?

La Same-Origin Policy du navigateur bloque par défaut les appels fetch() d'une extension Chrome ou de youtube.com vers un serveur externe. Le middleware CORS ajoute les en-têtes Access-Control-Allow-Origin/Methods nécessaires pour autoriser l'extension à appeler l'API.

3 - Comment gérer efficacement le chargement du modèle ?

Le pipeline .joblib est chargé une seule fois en mémoire au démarrage du module de l'API (variable globale), et non rechargé depuis le disque à chaque requête. Chaque appel /predict_batch réutilise l'objet déjà en RAM, ce qui garantit une latence faible (environ 4 ms pour 50 commentaires).

PHASE 5

Phase 5 : Développement de l'extension Chrome

1 - Comment gérer les commentaires avec des réponses (threads) ?

Mon content script extrait les commentaires principaux (ytd-comment-thread-renderer #content-text). Les réponses sont imbriquées dans ytd-comment-replies-renderer ; une amélioration serait d'ajouter une case 'Inclure les réponses' pour choisir d'analyser aussi les discussions.

2 - Quelle stratégie pour extraire les commentaires dynamiques ?

YouTube charge les commentaires en lazy-loading au défilement. L'extension lit les commentaires présents dans le DOM au moment de l'analyse (run_at document_idle) ; l'utilisateur fait défiler pour en charger davantage puis relance l'analyse. Un défilement automatique vers #comments est une amélioration prévue.

3 - Comment optimiser les performances avec de nombreux commentaires ?

Les commentaires visibles (plafonnés à 100 par extraction) sont envoyés en une seule requête /predict_batch et vectorisés en une passe. Les filtres par sentiment opèrent côté client, en mémoire dans popup.js, sans réinterroger le serveur.

PHASE 6

Phase 6 : Déploiement cloud avec Docker

1 - Pourquoi séparer les requirements dev et prod ?

requirements.txt contient les outils de dev (pytest, matplotlib, seaborn) ; requirements-prod.txt ne garde que l'exécution (fastapi, uvicorn, scikit-learn, pandas, numpy, joblib). L'image Docker n'installe que le prod, ce qui la rend plus légère et plus sûre.

2 - Comment réduire la taille de l'image Docker ?

Image de base légère (python:3.10-slim), installation des seuls requirements de production, copie uniquement des fichiers nécessaires (app, src, models), et --no-cache-dir pour pip. L'ordre des couches est optimisé pour profiter du cache.

3 - Quelle stratégie pour les mises à jour du modèle ?

Réentraîner via le pipeline reproductible, versionner les nouveaux .joblib et métriques, puis reconstruire et redéployer l'image. Un suivi des métriques permet de valider qu'une nouvelle version ne régresse pas avant mise en production.

4 - Cloud éphémère (Codespaces) vs hébergement PaaS permanent ?

Un Codespace est éphémère : il s'arrête après environ 30 min d'inactivité et son URL n'est active que lorsqu'il tourne, ce qui impose de le relancer avant chaque démonstration. Un PaaS permanent reste disponible en continu avec une URL stable, au prix d'un hébergement payant. Pour une vraie mise en production, le PaaS (ou un conteneur managé) est préférable pour la disponibilité.